

EXG: Self-Evolving Agents with Experience Graphs

Yuxin Jin
University of Technology Sydney
Sydney, Australia
yuxin.jin-1@student.uts.edu.au

Siyuan Zhang
University of Technology Sydney
Sydney, Australia
Siyuan.Zhang@uts.edu.au

Hanchen Wang
University of Technology Sydney
Sydney, Australia
hanchen.wang@uts.edu.au

Lu Qin
University of Technology Sydney
Sydney, Australia
Lu.Qin@uts.edu.au

Ying Zhang
University of Technology Sydney
Sydney, Australia
Ying.Zhang@uts.edu.au

Wenjie Zhang
The University of New South Wales
Sydney, Australia
wenjie.zhang@unsw.edu.au

Abstract

Large language model (LLM)-based agents have demonstrated strong capabilities in complex reasoning and problem solving through multi-step interactions, yet most deployed agents remain behaviorally static, with knowledge acquired during execution rarely translating into systematic improvement over time. In response, a growing line of work on self-evolving agents explores how agents can improve through experience during deployment, but most existing approaches either rely on ad hoc reflection limited to single-task correction or adopt unstructured memory that accumulates fragmented experience with delayed usability. To address this limitation, we introduce EXG, an experience graph framework for self-evolving agents that explicitly organizes accumulated successes and failures into a structured, relational representation. EXG is the first experience graph designed for self-evolving agents, supporting both online, real-time graph growth during execution for immediate cross-task experience reuse, and offline reuse of a consolidated experience graph as an external memory module. This design also enables EXG to serve as a plug-and-play component for existing self-evolving agents, organizing prior experience into a unified experience graph and improving both solution quality and resource efficiency as deployment progresses. Extensive experiments across code generation and reasoning benchmarks show that EXG attains more favorable performance-efficiency trade-offs than reflection- and memory-based baselines in both online and offline evaluations. Our results suggest that structuring experience as a graph provides a principled foundation for scalable and transferable self-evolving agent behavior.

1 Introduction

Recent advances in large language models (LLMs) have enabled agents to function as interactive problem solvers capable of sustaining long sequences of reasoning and action, making them suitable for tasks that involve extended interaction and non-trivial planning [23, 25, 37]. These advances have fueled rapid progress in autonomous agents across domains such as code generation and multi-hop question answering [4, 36]. Despite this progress, most deployed agents remain fundamentally static: each task is treated largely in isolation, and knowledge acquired during previous attempts rarely translates into systematic improvement on future tasks [16, 17, 21]. As a result, agents often repeat similar mistakes, fail to consolidate past successes, and incur growing computational costs as deployment continues [14, 19, 34].

To overcome the static nature of deployed agents, recent research has begun to explore self-evolving agents, which seek to improve future task performance by leveraging experience acquired through an agent’s own interactions. Rather than relying on parameter updates [29, 43], a prominent line of work studies non-parametric self-evolution in two settings, with online self-evolution [9, 19] generating and consuming experience within individual tasks during inference, and offline self-evolution [3, 11, 15, 48] accumulating experience across tasks and reusing it over time to guide subsequent reasoning. However, existing approaches along both directions face more fundamental limitations. Online self-evolving methods typically confine experience to a single task, lacking mechanisms to persist and reuse experience across tasks during inference. Offline self-evolving methods, while able to consolidate experience collected online and reuse it at test time, usually depend on time-consuming and complex post-processing. The experience generated during online interaction is often not directly usable. More broadly, self-evolving agents lack a unified framework for organizing experience that simultaneously supports online cross-task reuse and serves as an effective offline memory module for new tasks.

Motivated by these limitations, the design of experience graphs (EXG) is guided by three corresponding principles. Rather than confining experience to individual tasks, EXG externalizes interaction outcomes into a persistent experience graph, allowing knowledge acquired during one task to be immediately reused by subsequent tasks and enabling online self-evolution beyond task boundaries. At the same time, by maintaining a unified graph representation throughout deployment, EXG avoids the costly and ad hoc post-processing typically required by offline methods, making online experience directly reusable as an external memory at test time. EXG further provides a single, graph-centric abstraction for organizing experience that bridges online and offline self-evolution, while also establishing a plug-and-play foundation that allows existing self-evolving agents to incorporate structured experience without modifying their underlying architectures.

We propose EXG as a self-evolution mechanism that incrementally structures interaction experience during agent execution. Concretely, EXG abstracts each attempt within a task into a structured case and inserts it as a node in a growing experience graph, where edges encode relational signals such as task association, semantic similarity, and error correction. As the agent operates, newly generated experience is immediately integrated into this shared graph rather than remaining local to the current task. For each new task, EXG retrieves and reranks relevant cases from the graph

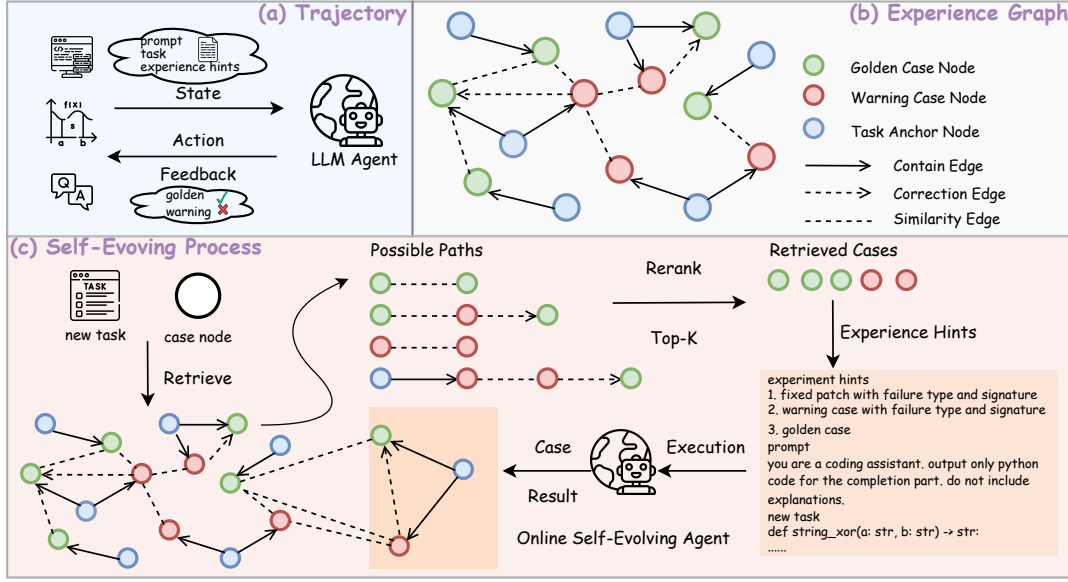


Figure 1: Overview of the self-evolving experience graph. (a) Trajectory produces structured cases from agent interactions. (b) Experience graph organizes cases into a relational structure. (c) Self-evolving process reuses retrieved experience across tasks and continuously incorporates resulting experience back into the graph.

to construct experience hints that guide reasoning, after which the resulting interaction is added back to the graph. This online self-evolving loop enables experience to accumulate and compound across tasks while remaining directly usable at inference time. Because a unified graph representation is maintained throughout deployment, the experience graph can be reused offline as an external memory module without additional consolidation or retraining, enabling EXG to bridge online and offline self-evolution within a single plug-and-play architecture. Figure 1 shows the construction of the experience graph and how experience is reused.

Our contributions can be summarized as follows:

- **Experience Graph for Self-Evolving Agents.** We introduce EXG, the first graph-based mechanism for self-evolving agents that explicitly encodes experience into a structured representation.
- **Unified Online and Offline Self-Evolution.** EXG bridges online self-evolution and cross-task experience reuse within a single representation by incrementally constructing the experience graph during execution and directly reusing the resulting graph as an external memory in offline settings.
- **Plug-and-Play Experience Organization.** By operating solely at inference time and remaining external to the agent’s internal reasoning mechanism, EXG serves as a plug-and-play module that can be seamlessly integrated into existing self-evolving agents, improving their ability to exploit accumulated experience without modifying underlying models.
- **Higher Accuracy at Lower Computational Cost.** We demonstrate through extensive experiments on reasoning and code generation benchmarks that structuring experience with EXG yields higher pass@1 exceeding **150%** and pass@2

approaching **30%** in the online setting and comparable performance in the offline setting, while substantially reducing interaction cost, with up to **45.7%** fewer LLM calls and up to **30.5%** lower LLM inference latency.

2 Experience Graph Design

2.1 Trajectories and Case Abstraction

An agent’s interaction with a task is naturally represented as a trajectory consisting of alternating reasoning, action, and feedback signals produced during execution, as illustrated in Figure 1 (a). Formally, for a given task τ , we denote an interaction trajectory as

$$\mathcal{T}_\tau = \{(s_t, a_t, o_t)\}_{t=1}^T, \quad (1)$$

where s_t represents the agent’s internal state or prompt context at step t , a_t denotes the action taken by the agent, and o_t is the observed outcome or feedback from the environment, such as execution results or correctness signals. The trajectory terminates after a finite number of steps when a solution is produced or a failure condition is reached. While this trajectory-level representation captures the full interaction process, it remains unstructured and task-local, making it unsuitable for direct reuse across tasks.

To enable structured reuse, EXG abstracts each completed attempt within a trajectory into a *case*, which serves as the atomic unit of experience in the graph. Specifically, given a trajectory $\mathcal{T}_\tau^{(k)}$ corresponding to the k -th attempt on task τ , we construct a case

$$c_\tau^{(k)} = (\tau, x_\tau, y_\tau^{(k)}, r_\tau^{(k)}, \sigma_\tau^{(k)}), \quad (2)$$

where x_τ denotes the task input, $y_\tau^{(k)}$ is the agent’s final output for this attempt, $r_\tau^{(k)} \in \{0, 1\}$ indicates whether the attempt is successful, and $\sigma_\tau^{(k)}$ summarizes salient execution signals extracted from

the trajectory, such as error messages, failure types, or corrective feedback. Successful cases ($r_\tau^{(k)} = 1$) are treated as *golden cases*, while unsuccessful ones ($r_\tau^{(k)} = 0$) are treated as *warning cases*. By collapsing a full interaction trajectory into a compact structured representation, cases provide a reusable abstraction that can be incrementally organized within the experience graph.

2.2 Experience Graph Construction

Based on the above case abstraction, EXG organizes accumulated experience into a relational experience graph, as illustrated in Figure 1 (b). Formally, the experience graph is defined as

$$\mathcal{G} = (\mathcal{V}, \mathcal{E}), \quad (3)$$

where the vertex set $\mathcal{V}$ consists of heterogeneous node types representing different levels of experience abstraction, and the edge set $\mathcal{E}$ encodes typed relations among them.

Nodes. The node set $\mathcal{V}$ includes two primary types:

Case nodes. Each case node corresponds to a case $c_\tau^{(k)}$ constructed from an interaction trajectory, as defined in Section 2.1. Case nodes, including *golden* and *warning* case nodes, serve as the atomic units of experience in the graph.

Task anchor nodes. For each task τ , a task anchor node a_τ is introduced to group all cases associated with the same task, providing a task-level entry point for organizing and retrieving experience.

Edges. Edges in the experience graph are typed and directed, capturing distinct semantic relations between nodes. We consider the following edge types:

Contain edges. A directed edge ($a_\tau \rightarrow c_\tau^{(k)}$) indicates that case $c_\tau^{(k)}$ is an attempt associated with task τ . These edges establish the hierarchical structure between task anchors and case nodes.

Similarity edges. An undirected edge ($c_i - c_j$) labeled as *similar_to* encodes semantic similarity between two cases. Similarity is computed based on representations derived from case attributes such as task inputs, prompts, or extracted signatures, enabling the graph to expose reusable patterns across different tasks.

Correction edges. For multiple attempts on the same task, a directed edge ($c_\tau^{(k)} \rightarrow c_\tau^{(k')}$) labeled as *fixed_by* indicates that the case $c_\tau^{(k')}$ corrects or resolves the failure observed in the earlier case $c_\tau^{(k)}$. These edges explicitly capture error-repair relationships within a task.

2.3 Experience Retrieval

Algorithm 1 summarizes the retrieval procedure. Given an experience graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, the retrieval process constructs a bounded pool of candidate cases to support the current attempt, corresponding to the *Retrieve* stage in Figure 1 (c). The retrieval process is conditioned on the *provisional case* c_q , a partially instantiated case that contains the task input and contextual information while its output and outcome fields remain undefined. Based on this provisional case, retrieval integrates three complementary sources, namely task-local context from *task anchor cases*, semantic neighborhoods induced by *similarity* edges, and corrective traces captured by *correction* edges.

Algorithm 1 Experience Retrieval

Require: Provisional case c_q , experience graph $\mathcal{G}$

Ensure: Candidate case set C

```

1:  $C_{\text{task}} \leftarrow$  cases attached to the task anchor of  $c_q$ 
2:  $\mathcal{A}_{\text{seed}} \leftarrow \text{Select}(C_{\text{task}})$ 
3:  $\mathcal{S}_{\text{query}} \leftarrow$  cases semantically related to  $c_q$   $\triangleright$  query-side seeds
4:  $\mathcal{S}_{\text{bridge}} \leftarrow$  cases reached by traversing similar_to edges from  $\mathcal{A}_{\text{seed}}$   $\triangleright$  task-side bridge seeds
5:  $\mathcal{S} \leftarrow \mathcal{S}_{\text{query}} \cup \mathcal{S}_{\text{bridge}}$ 
6:  $C_{\text{sim}} \leftarrow$  one-hop expansion from  $\mathcal{S}$  along similar_to edges
7:  $C_{\text{fix}} \leftarrow \emptyset$ 
8: for all  $c \in C_{\text{sim}}$  do
9:   if  $c$  has an outgoing correction edge then
10:    add the destination case to  $C_{\text{fix}}$ 
11:   end if
12: end for
13:  $C \leftarrow \text{Deduplicate}(C_{\text{task}} \cup C_{\text{sim}} \cup C_{\text{fix}})$ 
14: return  $\text{Cap}(C)$ 
```

Task-anchor cases. Let $a_{\tau(q)}$ denote the task anchor associated with c_q . The task-local set is

$$C_{\text{task}} = \{c \in \mathcal{V} \mid (a_{\tau(q)} \rightarrow c) \in \mathcal{E}_{\text{contain}}\}. \quad (4)$$

Semantic seeds and one-hop expansion. We form semantic seeds from two channels. The query-side seed set $\mathcal{S}_{\text{query}} \subseteq \mathcal{V}$ contains cases semantically related to c_q (e.g., via an embedding index). In addition, we select a subset of task-anchor cases as seeds for bridging, $\mathcal{A}_{\text{seed}} = \text{Select}(C_{\text{task}})$, where $\text{Select}(\cdot)$ prioritizes warning cases when available. Bridge seeds are then obtained by traversing *similar_to* relations from $\mathcal{A}_{\text{seed}}$,

$$\mathcal{S}_{\text{bridge}} = \{c' \in \mathcal{V} \mid \exists c \in \mathcal{A}_{\text{seed}}, (c \rightarrow c') \in \mathcal{E}_{\text{sim}}\}. \quad (5)$$

Let $\mathcal{S} = \mathcal{S}_{\text{query}} \cup \mathcal{S}_{\text{bridge}}$. A bounded semantic neighborhood is collected by one-hop expansion along *similarity* edges

$$C_{\text{sim}} = \{c' \in \mathcal{V} \mid \exists c \in \mathcal{S}, (c - c') \in \mathcal{E}_{\text{sim}}\}. \quad (6)$$

Corrective traces. To incorporate explicit error-repair relations, we follow *correction* edges from semantically expanded cases and add their corrected counterparts,

$$C_{\text{fix}} = \{c' \in \mathcal{V} \mid \exists c \in C_{\text{sim}}, (c \rightarrow c') \in \mathcal{E}_{\text{fix}}\}. \quad (7)$$

Final candidate pool. The final retrieval pool aggregates diverse and complementary signals by combining anchor cases from the current task, semantically similar cases retrieved via seed-based expansion, and associated corrective evidence, followed by deduplication and a global cap:

$$C = \text{Cap}(\text{Deduplicate}(C_{\text{task}} \cup C_{\text{sim}} \cup C_{\text{fix}})). \quad (8)$$

This design increases the likelihood that the retrieved pool offers rich semantic context and actionable experience, enabling more effective guidance for the current case.

2.4 Experience Reranking

Given the candidate pool C returned by retrieval, reranking orders cases by a relevance score that combines prompt-level similarity with failure-aware signals when available, and further incorporates structural proximity via one-hop propagation over *similarity* edges.

Case similarity. For each case c , we derive two embeddings, a prompt embedding $\mathbf{e}_p(c)$ from its prompt content and a failure embedding $\mathbf{e}_f(c)$ from its failure-related text. We also define an indicator $h(c) \in \{0, 1\}$ that denotes whether c contains failure information. The similarity between cases c_i and c_j is defined as

$$s(c_i, c_j) = \alpha \langle \mathbf{e}_p(c_i), \mathbf{e}_p(c_j) \rangle + (1 - \alpha) h(c_i) h(c_j) \langle \mathbf{e}_f(c_i), \mathbf{e}_f(c_j) \rangle, \quad (9)$$

where $\alpha \in [0, 1]$ controls the trade-off between prompt semantics and failure-aware similarity. The gate $h(c_i)h(c_j)$ ensures that the failure term contributes only when both cases carry failure signals.

Seed initialization. Reranking is initialized from a seed set $\mathcal{S}$ consisting of (i) query-side seeds semantically matched to the provisional case c_q and (ii) bridge seeds reached from warning-prioritized anchor-associated cases during retrieval. Each seed $c \in \mathcal{S}$ is assigned an initial relevance $\rho_0(c)$ reflecting its matching strength.

One-hop relevance propagation. To incorporate structural proximity, we propagate seed relevance through one-hop *similarity* edges. For candidate case $c \in \mathcal{C}$, we compute its final relevance as

$$\rho(c) = \max \left(\rho_0(c), \max_{u \in \mathcal{S}} [\rho_0(u) + w(u, c)] \right), \quad (10)$$

where $w(u, c)$ denotes the weight of the *similarity* edge from u to c in $\mathcal{E}$. Intuitively, a case receives a higher rank if it is either a strong seed itself or is one hop away from a strong seed via a high-affinity *similar_to* relation.

Ranking and selection. All candidates $c \in \mathcal{C}$ are sorted in descending order of $\rho(c)$. The score $\rho(\cdot)$ is used solely to order cases for selection and does not modify the experience graph $\mathcal{G}$.

Experience Hint Construction. Given the reranked case set $\mathcal{C}_{\text{rank}}$, EXG constructs a structured set of experience hints that summarizes relevant prior experience for the current attempt. Hint construction organizes cases by their semantic roles and selectively extracts salient information from each case.

Hint types from ranked cases. Given the reranked set $\mathcal{C}_{\text{rank}}$, EXG constructs three corresponding types of hints from the selected cases. Specifically, (i) *fixed-by hints* are formed when a case participates in a *fixed_by* relation and thus provides explicit error–repair information; (ii) *warning hints* are formed from unsuccessful cases to expose salient failure patterns; and (iii) *golden hints* are formed from successful cases to provide concise positive exemplars. The hint type determines both the information extracted from each case and its priority during hint assembly.

Hint assembly. Hints are assembled by iterating over $\mathcal{C}_{\text{rank}}$ in descending order of relevance and instantiating the corresponding hint type for each case, subject to a limited budget. Fixed-by hints are added with the highest priority, followed by warning hints and golden hints. The resulting ordered hint list is denoted by

$$\mathcal{H} = \{h_1, h_2, \dots, h_L\}. \quad (11)$$

Hint usage. The constructed hint set $\mathcal{H}$ is provided as auxiliary context for the current attempt as shown in Figure 2.

3 Self-Evolution with Experience Graphs

3.1 Online Self-Evolution

Online self-evolution in EXG is realized through a closed-loop interaction between agent execution and the experience graph, as illustrated in Figure 1 (c). For each incoming task τ , the agent

System:
You are a coding assistant. Output ONLY Python code for the completion part. Do not include explanations.
User:
Complete the following Python function: <problem_prompt>
=== MEMORY HINTS (via EXG) ===
<structured hints from past cases>
Return only the code that should be appended after the prompt.

Figure 2: EXG structured prompt architecture.

first initializes a provisional case c_q , which contains the task input and contextual information but does not yet include an output or correctness outcome. Conditioned on c_q , the experience graph $\mathcal{G}$ is queried to retrieve relevant prior cases using the graph retrieval and reranking procedures defined in the EXG design.

Based on the reranked cases, EXG constructs a set of structured experience hints, which are injected into the agent’s prompt to guide the current attempt. The agent then executes the task under the guidance of these hints and produces an output. After execution completes, the provisional case c_q is finalized into a complete case by attaching the generated output, the correctness signal, and extracted execution signatures, yielding a new case $c_\tau^{(k)}$.

The finalized case is then incorporated back into the experience graph according to the graph update rules. Specifically, a new case node corresponding to $c_\tau^{(k)}$ is added to $\mathcal{G}$ and connected to the task anchor node a_τ via a *contain* edge. If the new case corresponds to a corrective attempt following an earlier failure on the same task, a directed *fixed_by* edge is added to explicitly encode the error–repair relation. In addition, similarity relations between the new case and existing cases are established through *similarity* edges based on their semantic affinity.

Through this self-evolving loop, experience generated during online interaction is incrementally externalized into the experience graph and becomes immediately available for subsequent tasks. As a result, online self-evolution in EXG enables experience to compound across tasks during deployment, while operating entirely at inference time without modifying the model parameters of agents.

3.2 Offline Reuse

In addition to online self-evolution, EXG naturally supports an offline setting through reuse of a previously constructed experience graph. In this regime, the experience graph is built in advance from a set of training tasks and then held fixed during evaluation on unseen tasks, as illustrated in Figure 3.

The offline setting differs from the online setting only in how the graph is deployed, rather than in how experience is represented or exploited. The same case abstraction, graph structure, and inference-time pipeline—including experience retrieval, relevance propagation, reranking, and hint construction—are used without modification, ensuring that experience is accessed and injected in a consistent manner across settings.

The key distinction is that no graph updates are performed in the offline setting. During evaluation, newly generated cases are not added to the graph, and no new similarity or corrective edges are created. As a result, the agent benefits from accumulated experience

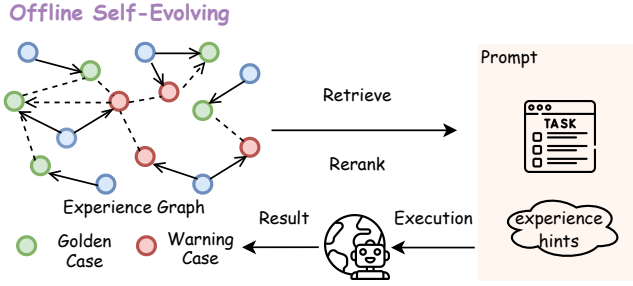


Figure 3: Offline self-evolving via graph reuse. EXG is pre-constructed during online process and held fixed.

encoded in the frozen graph while preventing information leakage from test tasks.

By sharing a unified abstraction and inference-time pipeline across online and offline regimes, EXG provides a single experience-centric framework that supports both continual accumulation and static reuse, differing only in deployment rather than in mechanism.

3.3 Plug-and-Play Integration

Beyond serving as a standalone framework, EXG is designed as a plug-and-play experience module that can be integrated into a wide range of existing self-evolving agents. This property arises from the fact that EXG operates entirely at inference time and externalizes experience into a structured graph, without requiring changes to the underlying language model or task-specific solving procedures. As a result, EXG can be attached to different agent architectures as a shared experience layer, rather than as a tightly coupled algorithmic component.

In practice, EXG can augment both online and offline self-evolving methods by providing a persistent and structured experience foundation. For online agents, experience signals such as reflections or corrective feedback can be accumulated and reused across tasks, reducing repeated LLM calls for rediscovery. For offline agents, experience collected during training can be organized into the same graph structure and later accessed during evaluation through graph-based retrieval. In both cases, EXG enables experience to generalize across tasks via semantic and relational organization, offering a unified and reusable experience backbone that improves effectiveness while lowering inference-time cost.

4 Experiments

4.1 Experimental Setup

4.1.1 Datasets and Models. We evaluate EXG on a diverse set of benchmarks that cover program synthesis and multi-hop question answering, representing different forms of sequential reasoning and decision-making. Concretely, our evaluation includes HumanEval [4], EvalPlus [10], MuSiQue [20] and HotpotQA [36], each reflecting distinct problem settings and reasoning demands. We conduct experiments using multiple sizes of the Qwen3 [33] model family. More details can be found in appendix B.

4.1.2 Experimental Protocol and Hyperparameters. Each task is allowed up to two attempts, consisting of a single initial attempt

followed by at most one retry. Given a task query, EXG retrieves candidate seeds using dense vector similarity via a FAISS index. Specifically, the top- K_s most similar cases ($K_s = 10$) are first retrieved using sentence-level embeddings computed with a MiniLM encoder [22]. These seeds are then expanded through the experience graph by one-hop traversal over *similarity* edges with a fanout $F_{sim} = 5$. Each task is associated with an anchor node, from which at most one directly related case is selected, followed by similarity-based bridge expansion with a fanout $F_{bridge} = 5$. After merging and deduplication, the total number of candidate cases is globally capped at $K_c = 30$. Candidate relevance is computed by combining prompt-level similarity and failure-aware similarity using a weighting factor $\alpha = 0.8$. From the ranked candidates, EXG constructs at most $H = 5$ experience hints, which are injected into the model prompt. Effectiveness is evaluated using pass@1 and pass@2, while efficiency is measured by the average number of LLM calls, LLM inference latency, and retrieval latency.

4.2 Online Experience Graph Performance

4.2.1 Baselines. We compare EXG against representative online self-evolving agents, including Reflexion and SE-Agent. Reflexion [19] augments the agent with self-generated verbal feedback, enabling iterative correction by reflecting on previous failures, while SE-Agent [9] performs trajectory-level self-evolution through operations such as revision and recombination across multiple reasoning paths. Since the benchmarks considered in this work typically involve short interaction horizons and limited trajectories, we additionally include a simplified variant of SE-Agent that only applies the revision operation denoted as SE-Agent-Rev, in order to provide a more comparable baseline under constrained retry budgets.

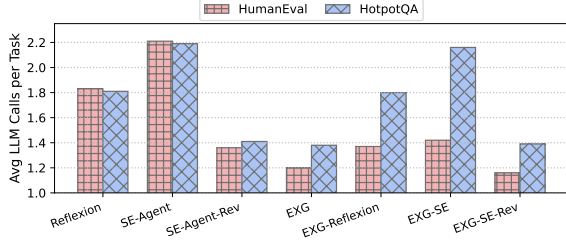
To ensure a fair comparison under identical model interaction constraints, we further integrate EXG into these online self-evolving frameworks in a plug-and-play manner, resulting in EXG-Reflexion and EXG-SE variants. These settings allow us to evaluate whether explicitly structuring experience as a graph can complement existing online self-evolution mechanisms without increasing the number of model calls or modifying the underlying agent logic.

4.2.2 Results. Table 1 shows that EXG consistently strengthens online self-evolution across both code generation and multi-hop reasoning tasks. Compared with Reflexion and SE-Agent variants, EXG together with its plug-and-play extensions achieve higher pass@1 in nearly all settings, indicating more effective first-attempt reasoning. These improvements generally translate to higher pass@2 as well, with the best pass@2 results often achieved by EXG-based methods, suggesting that accumulated experience not only reduces repeated errors but also increases the utility of the limited retry budget. Importantly, these gains hold across model scales, from compact models to stronger code- and reasoning backbones.

On HumanEval and EvalPlus, EXG delivers the largest relative gains on smaller and mid-sized models, where prior experience can most directly compensate for limited model capacity. For example, on HumanEval with Qwen3-1.7B, EXG improves pass@1 by more than **150%** relative to Reflexion, while EXG-Reflexion further increases pass@2 by roughly **30%**. As model capacity increases, the relative gains narrow but remain consistent: with Qwen3-Coder-Flash, EXG still improves pass@1 by over **30%** compared to the

Table 1: Online performance comparison across baselines and EXG, reported in terms of pass@1 and pass@2 on different datasets and models.

Dataset	Model	Reflexion		SE-Agent		SE-Agent-Rev		EXG		EXG-Reflexion		EXG-SE		EXG-SE-Rev	
		p@1	p@2	p@1	p@2	p@1	p@2	p@1	p@2	p@1	p@2	p@1	p@2	p@1	p@2
HumanEval	Qwen3-1.7B	0.207	0.543	0.195	0.524	0.207	0.561	0.537	0.585	0.573	0.695	0.512	0.610	0.537	0.598
	Qwen3-8B	0.201	0.610	0.177	0.720	0.189	0.823	0.415	0.488	0.506	0.720	0.573	0.835	0.591	0.799
	Qwen3-Coder-Flash	0.585	0.780	0.598	0.939	0.640	0.945	0.805	0.835	0.817	0.872	0.860	0.951	0.835	0.921
EvalPlus	Qwen3-1.7B	0.134	0.342	0.189	0.207	0.207	0.482	0.323	0.439	0.317	0.494	0.408	0.415	0.299	0.445
	Qwen3-8B	0.183	0.274	0.317	0.341	0.341	0.750	0.409	0.530	0.390	0.598	0.524	0.537	0.311	0.677
	Qwen3-Coder-Flash	0.585	0.872	0.610	0.768	0.591	0.848	0.811	0.866	0.793	0.866	0.793	0.848	0.817	0.915
MuSiQue	Qwen3-14B	0.174	0.326	0.356	0.700	0.382	0.836	0.320	0.406	0.480	0.748	0.490	0.736	0.560	0.896
	Qwen-Plus	0.542	0.940	0.552	0.840	0.568	0.970	0.608	0.672	0.652	0.946	0.652	0.854	0.694	0.990
	Qwen-Max	0.346	0.692	0.348	0.746	0.370	0.882	0.446	0.516	0.478	0.762	0.486	0.926	0.518	0.924
HotpotQA	Qwen3-8B	0.283	0.494	0.283	0.410	0.275	0.344	0.460	0.500	0.462	0.557	0.445	0.516	0.458	0.509
	Qwen-Plus	0.597	0.662	0.603	0.670	0.587	0.669	0.618	0.642	0.601	0.674	0.613	0.681	0.608	0.680
	Qwen-Max	0.616	0.686	0.607	0.669	0.621	0.680	0.659	0.684	0.651	0.686	0.657	0.707	0.655	0.703

**Figure 4: Average number of LLM calls per task under the online setting on HumanEval and HotpotQA.**

strongest non-graph baseline. On EvalPlus, which applies stricter correctness checks, EXG maintains comparable or larger relative improvements in pass@1 while also improving pass@2, indicating that the benefits are not due to overfitting shallow test cases but reflect more robust semantic correctness.

On MuSiQue and HotpotQA, EXG exhibits a different but complementary pattern. On MuSiQue, which emphasizes compositional reasoning, EXG substantially boosts pass@1, with relative improvements of around **40–50%** on Qwen3-14B and close to **30%** on Qwen-Plus, while also yielding consistent relative gains in pass@2 across model variants. This suggests that graph-structured experience helps abstract reusable reasoning patterns across related question compositions. On HotpotQA, where question diversity and reasoning paths are less regular, gains are smaller but stable: EXG improves pass@1 by over **60%** on Qwen3-8B and by around **7%** on Qwen-Max. Taken together, these results indicate that EXG is particularly effective when tasks share latent structural regularities, while still providing consistent first-attempt improvements in more heterogeneous reasoning settings.

4.3 Online Efficiency Analysis

4.3.1 LLM Calls. Figure 4 illustrates the average number of LLM calls per task for different online methods. On HumanEval, the base EXG method requires 1.20 calls per task, compared to 1.83 for Reflexion and 2.21 for SE-Agent, corresponding to reductions of **34.4%** and **45.7%**, respectively. EXG also consistently outperforms revised baselines, reducing LLM calls from 1.36 to 1.20, a relative decrease of **11.8%**. On HotpotQA, the reduction remains consistent with EXG lowering LLM calls from 2.19 for the most expensive baseline to 1.38, achieving a **37%** reduction.

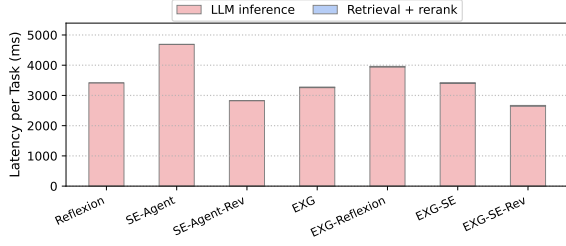
Overall, EXG-based methods require fewer LLM calls than their corresponding online baselines, indicating that the observed performance improvements are achieved with lower interaction cost rather than increased model usage. This reduction primarily stems from improved first-attempt success rates: unlike Reflexion or SE-Agent, which invoke the LLM multiple times for reflection or revision after failures, EXG provides richer guidance at inference time through structured experience reuse, thereby avoiding repeated model calls.

This trend also holds for plug-and-play variants. On HumanEval, EXG-Reflexion reduces the average number of LLM calls from **1.83** to **1.37**, while EXG-SE-Rev achieves the lowest cost at **1.16** calls per task. On HotpotQA, plug-and-play variants likewise incur fewer LLM calls than their original counterparts, although the reductions are more moderate due to the greater diversity of question types and reasoning paths.

4.3.2 Latency. Figure 5 reports the latency breakdown of different online methods on HumanEval, decomposing per-task runtime into LLM inference time and auxiliary overhead. In general, EXG-based methods achieve lower or comparable end-to-end latency, while the retrieval overhead introduced by the experience graph is minimal (**~18–22 ms**). Concretely, EXG reduces the average LLM latency to 3,259 ms, compared to 3,416 ms for Reflexion and 4,689 ms for SE-Agent, corresponding to latency reductions of **4.6%** and **30.5%**, respectively, by avoiding repeated reflection calls. When combined

Table 2: Offline performance comparison between ExpeL and EXG-Reflexion. Results include collection-stage pass@1 and pass@2 (C-p@1 and C-p@2, resp.) with average LLM calls, and test-stage pass@1 and pass@2 (T-p@1 and T-p@2, resp.).

Benchmark	ExpeL					EXG-Reflexion				
	C-p@1	C-p@2	Avg. Calls	T-p@1	T-p@2	C-p@1	C-p@2	Avg. Calls	T-p@1	T-p@2
HumanEval (Qwen3-Coder-Flash)	0.573	0.771	1.85	0.909	0.909	0.824	0.885	1.35	0.879	0.879
HotpotQA (Qwen-Plus)	0.600	0.661	1.80	0.590	0.633	0.614	0.670	1.79	0.610	0.637

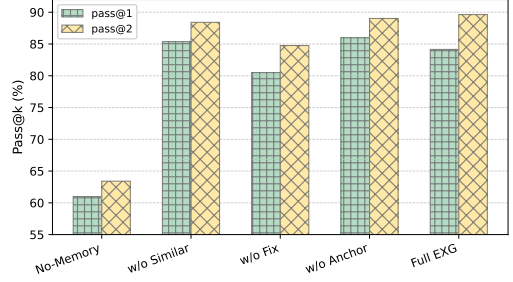
**Figure 5: Latency breakdown under the online setting on HumanEval, showing LLM inference and retrieval overhead.**

with reflection, EXG-Reflexion incurs slightly higher latency than Reflexion alone (3,943 ms vs. 3,416 ms) due to the additional reflection step. For SE-Agent, EXG consistently reduces latency across variants: EXG-SE lowers LLM inference time from 4,689 ms to 3,399 ms (27.5% reduction), while EXG-SE-Rev further reduces latency relative to SE-Agent-Rev from 2,826.6 ms to 2,648 ms, corresponding to a 6.3% reduction. These results indicate that EXG effectively reduces reflection-heavy computation while introducing only negligible retrieval overhead.

4.4 Offline Experience Graph Performance

4.4.1 Baselines. We adopt ExpeL [48] as the representative offline self-evolving baseline and compare it with EXG-Reflexion under a matched two-stage protocol for fair comparison. ExpeL performs experience collection in an online phase using reflexion-based retries, and in the offline phase it processes the collected data, particularly the reflection contents, to abstract a set of high-level insights. To align with this mechanism, we choose the reflexion variant of EXG for offline comparison for reasons below. (1) EXG-Reflexion also relies on reflection-based signals during online collection. (2) In the offline stage, the experience graph constructed is kept fixed, and insights are extracted from the graph together with the associated reflections, matching ExpeL’s offline abstraction process. At test time, each method follows its own retrieval strategy: ExpeL injects a fixed memory consisting of five insights and five golden cases, whereas EXG-Reflexion injects five insights together with five experience hints retrieved from the graph, which may include golden, warning, and fix-related cases. For each benchmark, the dataset is randomly shuffled and split into a 7:3 ratio for online collection and offline evaluation, respectively.

4.4.2 Results. As shown in Table 2, during the online collection stage, EXG-Reflexion consistently achieves stronger performance

**Figure 6: Structural ablation study of EXG on HumanEval under different graph configurations.**

with lower interaction cost than ExpeL. On HumanEval, EXG-Reflexion improves collection-stage pass@1 from **0.573** to **0.824** and pass@2 from **0.771** to **0.885**, while reducing the average number of LLM calls from **1.85** to **1.35**. This substantial gain indicates that the experience graph enables EXG-Reflexion to solve a larger fraction of tasks on the first attempt, significantly reducing reliance on repeated reflections during online collection. In the offline test stage with a frozen memory, the two methods exhibit comparable performance on HumanEval. EXG-Reflexion attains a test-stage pass@1 of **0.879**, slightly below ExpeL’s **0.909**. This gap can be partially attributed to the reduced volume of reflection traces collected online by EXG-Reflexion: because EXG more effectively leverages experience during online interaction, fewer reflection calls are triggered, resulting in fewer reflection-derived insights available for offline reuse. On HotpotQA, the task distribution is less concentrated and experience reuse is inherently more challenging. During online collection, EXG-Reflexion achieves slightly higher pass@1 (**0.614** vs. **0.600**) and pass@2 (**0.670** vs. **0.661**) than ExpeL, while incurring a comparable number of LLM calls (**1.79** vs. **1.80**). In the offline test stage, EXG-Reflexion continues to outperform ExpeL, achieving pass@1 of **0.610** and pass@2 of **0.637**, compared to **0.590** and **0.633**, respectively. In summary, these results indicate that EXG can effectively construct a reusable and generalizable experience graph during online interaction, which supports competitive and, in some cases, improved performance in offline evaluation despite operating under a frozen-memory setting.

4.5 Ablation Study

Table 3: Hint type ablation study of EXG on HumanEval.

Group	pass@1	pass@2
No-Memory	0.610	0.640
Similar-Only	0.823	0.835
Warning-Only	0.866	0.884
EXG	0.872	0.896

4.5.1 Structural Ablations of Experience Graph. We conduct a structural ablation study to examine the contribution of different components in the experience graph. Specifically, we compare the full EXG against several variants that remove individual structural elements.

No-Memory disables experience graph construction entirely.

w/o Similar removes the *similarity* edges from the graph and replaces them with standard query-based similarity retrieval.

w/o Fix removes the *correction* edges, eliminating explicit error-repair links between failed and successful cases.

w/o Anchor removes task-level anchor nodes, forcing retrieval to rely solely on case-level relations without task-centric grouping.

Full EXG retains all node and edge types, including task anchors, *similarity* edges, and *correction* edges.

As shown in Figure 6, the full EXG consistently achieves the best performance, while No-Memory performs worst due to the absence of structured experience. Among the ablated variants, removing *correction* edges leads to the largest performance drop, indicating that explicit corrective information from failures to successful solutions provides a stable and significant benefit. Removing *similarity* edges also degrades performance, suggesting that similarity relations play an important role in retrieving relevant cases. Finally, removing anchor nodes results in a smaller but consistent decrease compared to the full EXG, showing that task-level anchoring and bridge-based expansion further improve retrieval effectiveness. Together, these results demonstrate that the performance gains of EXG arise from the joint contribution of multiple graph components, with *fixed_by* relations providing the most critical signal.

4.5.2 Hint Type Ablation. We further conduct a hint-type ablation study to analyze the impact of different compositions of experience hints. As shown in Table 3, No-Memory serves as a reference without experience injection, while Similar-Only injects only golden cases retrieved by similarity, providing correct but limited guidance. Warning-Only further enriches the injected information by incorporating warning cases that encode failure types and signatures, enabling the agent to reason about potential mistakes. The full EXG setting injects a heterogeneous set of experience hints, including corrective signals derived from *fixed_by* relations in addition to golden and warning information. The results show a clear performance improvement as the injected experience becomes more diverse, with EXG achieving the best pass@1 and pass@2. This indicates that, under the EXG framework, correct solutions, failure-aware signals, and explicit corrective information each provide complementary benefits, and their joint composition is crucial for effective experience reuse.

5 Related Work

5.1 Self-Evolving Agents

Self-evolving agents seek to improve their behavior over time by leveraging experience generated through their own interactions, rather than remaining functionally fixed. Prior work in this area can be broadly categorized by whether self-evolution is achieved through updating model parameters or without modifying them. Parameter-updating approaches [13, 29, 30, 39, 43, 45] internalize experience via iterative training or reinforcement learning, enabling continual improvement at the cost of retraining overhead and reduced modularity. In contrast, non-parameter methods realize self-evolution at inference time and can be further divided based on how experience is utilized. Online approaches [9, 19, 40] primarily rely on reflexion or trajectory-level optimization to correct behavior within a single task, resulting in task-local improvements without persistent experience reuse. Offline approaches [2, 3, 6, 8, 11, 12, 35, 44, 46, 48] instead accumulate experience across tasks and apply consolidation or abstraction before reuse, enabling cross-task improvement through external memory or experience repositories. Memory-based methods highlight the role of persistent experience in self-evolution. However, they typically use flat or weakly structured representations and require substantial offline refinement before reuse.

5.2 Agent Memory Systems

Agent memory systems investigate how external memory can be incorporated to support long-horizon interaction while improving consistency and efficiency in LLM-based agents. A subset of this line of work [24, 49] is closely related to memory-based approaches in self-evolving agents, since they also reuse past trajectories, experiences, or reasoning traces across tasks; however, such methods primarily treat memory as an auxiliary information source for task execution, focusing on retrieval and prompt injection rather than modeling how an agent’s behavior systematically evolves with accumulated experience. Beyond this overlap, agent memory systems constitute a broader and more mature research direction, encompassing methods that model persistent context or knowledge for long-term coherence [5, 26, 28, 32, 38], treat memory as a system-level or control component governing memory organization and lifecycle [7, 18, 41, 42], and conduct efficiency analysis and benchmarking to examine the computational trade-offs and behavioral effects of memory in continual settings [1, 27, 31, 47]. Accordingly, approaches that aim to explicitly model experience-driven agent evolution do not fall within the scope of memory systems, though techniques developed in this literature provide valuable insights that can be leveraged to support more effective self-evolving agents.

6 Conclusion

In this work, we introduced EXG, an experience graph framework that enables self-evolving agents to systematically accumulate, organize, and reuse experience during deployment without modifying model parameters. By externalizing experience into a structured graph and operating entirely at inference time, EXG provides a principled mechanism for transforming ephemeral trial-and-error interactions into reusable knowledge. Extensive experiments across

online and offline settings demonstrate that EXG improves both effectiveness and efficiency: it achieves higher task success rates with fewer model calls during online interaction and retains competitive performance when reused as a frozen experience module offline. Moreover, EXG reduces overall LLM inference time by mitigating unnecessary retries and reflections, while the overhead introduced by graph-based retrieval remains marginal. Through comprehensive ablation studies, we further show that EXG’s benefits arise from both its graph structure and its ability to compose heterogeneous experience signals in a type-aware manner. In addition, EXG can be readily incorporated into existing self-evolving agents as an external experience layer, consistently enhancing performance without requiring changes to the underlying agent architecture. Together, these results suggest that structuring experience as a graph offers a scalable and modular foundation for self-evolving agents, enabling improvements to compound over time rather than reset across tasks.

References

- [1] Qingyao Ai, Yichen Tang, Changyue Wang, Jianming Long, Weihang Su, and Yiqun Liu. 2025. MemoryBench: A Benchmark for Memory and Continual Learning in LLM Systems. arXiv:2510.17281 [cs.LG] <https://arxiv.org/abs/2510.17281>
- [2] Zhicheng Cai, Xinyuan Guo, Yu Pei, Jiangtao Feng, Jinsong Su, Jiangjie Chen, Ya-Qin Zhang, Wei-Ying Ma, Mingxuan Wang, and Hao Zhou. 2025. FLEX: Continuous Agent Evolution via Forward Learning from Experience. arXiv:2511.06449 [cs.LG] <https://arxiv.org/abs/2511.06449>
- [3] Zouying Cao, Jiaji Deng, Li Yu, Weikang Zhou, Zhaoyang Liu, Bolin Ding, and Hai Zhao. 2025. Remember Me, Refine Me: A Dynamic Procedural Memory Framework for Experience-Driven Agent Evolution. arXiv:2512.10696 [cs.AI] <https://arxiv.org/abs/2512.10696>
- [4] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374 [cs.LG] <https://arxiv.org/abs/2107.03374>
- [5] Jizhan Fang, Xinle Deng, Haoming Xu, Ziyang Jiang, Yuqi Tang, Ziwen Xu, Shumin Deng, Yunzhi Yao, Mengru Wang, Shuofei Qiao, Huajun Chen, and Ningyu Zhang. 2025. LightMem: Lightweight and Efficient Memory-Augmented Generation. arXiv:2510.18866 [cs.CL] <https://arxiv.org/abs/2510.18866>
- [6] Jackson Hassell, Dan Zhang, Hannah Kim, Tom Mitchell, and Estevam Hruschka. 2025. Learning from Supervision with Semantic and Episodic Memory: A Reflective Approach to Agent Adaptation. arXiv:2510.19897 [cs.CL] <https://arxiv.org/abs/2510.19897>
- [7] Chuanrui Hu, Xingze Gao, Zuyi Zhou, Dannong Xu, Yi Bai, Xintong Li, Hui Zhang, Tong Li, Chong Zhang, Lidong Bing, and Yafeng Deng. 2026. EverMemOS: A Self-Organizing Memory Operating System for Structured Long-Horizon Reasoning. arXiv:2601.02163 [cs.AI] <https://arxiv.org/abs/2601.02163>
- [8] Xuechen Liang, Meiling Tao, Yinghui Xia, Jianhui Wang, Kun Li, Yijin Wang, Yangfan He, Jingsong Yang, Tianyu Shi, Yuntao Wang, Miao Zhang, and Xueqian Wang. 2025. SAGE: Self-evolving Agents with Reflective and Memory-augmented Abilities. *Neurocomput.* 647, C (Sept. 2025), 12 pages. doi:10.1016/j.neucom.2025.130470
- [9] Jiaye Lin, Yifu Guo, Yuzhen Han, Sen Hu, Ziyi Ni, Licheng Wang, Mingguang Chen, Hongzhang Liu, Ronghao Chen, Yangfan He, Daxin Jiang, Binxing Jiao, Chen Hu, and Huacan Wang. 2025. SE-Agent: Self-Evolution Trajectory Optimization in Multi-Step Reasoning with LLM-Based Agents. arXiv:2508.02085 [cs.AI] <https://arxiv.org/abs/2508.02085>
- [10] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and LINGMING ZHANG. 2023. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 21558–21572. https://proceedings.neurips.cc/paper_files/paper/2023/file/43e9d647ccd3e4b7b5baab53f0368686-Paper-Conference.pdf
- [11] Yitao Liu, Chenglei Si, Karthik R Narasimhan, and Shunyu Yao. 2025. Contextual Experience Replay for Self-Improvement of Language Agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 14179–14198. doi:10.18653/v1/2025.acl-long.694
- [12] Zibin Liu, Cheng Zhang, Xi Zhao, Yunfei Feng, Bingyu Bai, Dahu Feng, Erhu Feng, Yubin Xia, and Haibo Chen. 2025. Beyond Training: Enabling Self-Evolution of Agents with MOBIMEM. arXiv:2512.15784 [cs.AI] <https://arxiv.org/abs/2512.15784>
- [13] Hongliang Lu, Yuhang Wen, Pengyu Cheng, Ruijin Ding, Jiaqi Guo, Haotian Xu, Chutian Wang, Haonan Chen, Xiaoxi Jiang, and Guanjin Jiang. 2025. Search Self-play: Pushing the Frontier of Agent Capability without Supervision. arXiv:2510.18821 [cs.LG] <https://arxiv.org/abs/2510.18821>
- [14] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 46534–46594. https://proceedings.neurips.cc/paper_files/paper/2023/file/91edff07232fb1b55a505a9e9f6c0ff3-Paper-Conference.pdf
- [15] Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Ruijun Han, Long T. Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. 2025. ReasoningBank: Scaling Agent Self-Evolving with Reasoning Memory. arXiv:2509.25140 [cs.AI] <https://arxiv.org/abs/2509.25140>
- [16] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2024. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560 [cs.AI] <https://arxiv.org/abs/2310.08560>
- [17] Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology* (San Francisco, CA, USA) (UIST ’23). Association for Computing Machinery, New York, NY, USA, Article 2, 22 pages. doi:10.1145/3586183.3606763
- [18] Hongjin Qian, Zhao Cao, and Zheng Liu. 2026. MemoBrain: Executive Memory as an Agentic Brain for Reasoning. arXiv:2601.08079 [cs.AI] <https://arxiv.org/abs/2601.08079>
- [19] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 8634–8652. https://proceedings.neurips.cc/paper_files/paper/2023/file/1b44b878bb782e6954cd888628510e90-Paper-Conference.pdf
- [20] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. MuSiQue: Multihop Questions via Single-hop Question Composition. *Transactions of the Association for Computational Linguistics* 10 (2022), 539–554. doi:10.1162/tacl_a_00475
- [21] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv:2305.16291 [cs.AI] <https://arxiv.org/abs/2305.16291>
- [22] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. 2020. MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 5776–5788. https://proceedings.neurips.cc/paper_files/paper/2020/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf
- [23] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. arXiv:2203.11171 [cs.CL] <https://arxiv.org/abs/2203.11171>
- [24] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2024. Agent Workflow Memory. arXiv:2409.07429 [cs.CL] <https://arxiv.org/abs/2409.07429>
- [25] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) (NIPS ’22). Curran Associates Inc., Red Hook, NY, USA, Article 1800, 14 pages.
- [26] Rubin Wei, Jiaqi Cao, Jiarui Wang, Jushi Kai, Qipeng Guo, Bowen Zhou, and Zhouhan Lin. 2025. MLP Memory: A Retriever-Pretrained Memory for Large Language Models. arXiv:2508.01832 [cs.CL] <https://arxiv.org/abs/2508.01832>

- [27] Tianxin Wei, Naveen Sachdeva, Benjamin Coleman, Zhankui He, Yuanchen Bei, Xuying Ning, Mengting Ai, Yunzhe Li, Jingrui He, Ed H. Chi, Chi Wang, Shuo Chen, Fernando Pereira, Wang-Cheng Kang, and Derek Zhiyuan Cheng. 2025. Evo-Memory: Benchmarking LLM Agent Test-time Learning with Self-Evolving Memory. arXiv:2511.20857 [cs.CL] <https://arxiv.org/abs/2511.20857>
- [28] Rebecca Westhäußer, Wolfgang Minker, and Sebastian Zepf. 2025. Enabling Personalized Long-term Interactions in LLM-based Agents through Persistent Memory and User Profiles. arXiv:2510.07925 [cs.AI] <https://arxiv.org/abs/2510.07925>
- [29] Rong Wu, Xiaoman Wang, Jianbiao Mei, Pinlong Cai, Daocheng Fu, Cheng Yang, Licheng Wen, Xuemeng Yang, Yufan Shen, Yuxin Wang, and Botian Shi. 2025. EvolveR: Self-Evolving LLM Agents through an Experience-Driven Lifecycle. arXiv:2510.16079 [cs.CL] <https://arxiv.org/abs/2510.16079>
- [30] Peng Xia, Kaide Zeng, Jiaqi Liu, Can Qin, Fang Wu, Yiyang Zhou, Caiming Xiong, and Huaxiu Yao. 2025. Agent0: Unleashing Self-Evolving Agents from Zero Data via Tool-Integrated Reasoning. arXiv:2511.16043 [cs.LG] <https://arxiv.org/abs/2511.16043>
- [31] Zidi Xiong, Yuping Lin, Wenya Xie, Pengfei He, Zirui Liu, Jiliang Tang, Himabindu Lakkaraju, and Zhen Xiang. 2025. How Memory Management Impacts LLM Agents: An Empirical Study of Experience-Following Behavior. arXiv:2505.16067 [cs.AI] <https://arxiv.org/abs/2505.16067>
- [32] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-MEM: Agentic Memory for LLM Agents. arXiv:2502.12110 [cs.CL] <https://arxiv.org/abs/2502.12110>
- [33] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] <https://arxiv.org/abs/2505.09388>
- [34] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. 2024. Large Language Models as Optimizers. In *International Conference on Learning Representations*, B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun (Eds.), Vol. 2024. 12028–12068. https://proceedings.iclr.cc/paper_files/paper/2024/file/3339f19c5fcee3ad74502947a32be9e6-Paper-Conference.pdf
- [35] Cheng Yang, Xuemeng Yang, Licheng Wen, Daocheng Fu, Jianbiao Mei, Rong Wu, Pinlong Cai, Yufan Shen, Nianchen Deng, Botian Shi, Yu Qiao, and Haifeng Li. 2025. Learning on the Job: An Experience-Driven Self-Evolving Agent for Long-Horizon Tasks. arXiv:2510.08002 [cs.CL] <https://arxiv.org/abs/2510.08002>
- [36] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun'ichi Tsujii (Eds.). Association for Computational Linguistics, Brussels, Belgium, 2369–2380. doi:10.18653/v1/D18-1259
- [37] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629 [cs.CL] <https://arxiv.org/abs/2210.03629>
- [38] Yi Yu, Liuyi Yao, Yuexiang Xie, Qingquan Tan, Jiaqi Feng, Yaliang Li, and Libing Wu. 2026. Agentic Memory: Learning Unified Long-Term and Short-Term Memory Management for Large Language Model Agents. arXiv:2601.01885 [cs.CL] <https://arxiv.org/abs/2601.01885>
- [39] Zhenrui Yue, Kartikeya Upasani, Xianjun Yang, Suyu Ge, Shaoliang Nie, Yuning Mao, Zhe Liu, and Dong Wang. 2026. Dr. Zero: Self-Evolving Search Agents without Training Data. arXiv:2601.07055 [cs.AI] <https://arxiv.org/abs/2601.07055>
- [40] Yunpeng Zhai, Shuchang Tao, Cheng Chen, Anni Zou, Ziqian Chen, Qingxu Fu, Shinji Mai, Li Yu, Jiaji Deng, Zouying Cao, Zhaoyang Liu, Bolin Ding, and Jingren Zhou. 2025. AgentEvolver: Towards Efficient Self-Evolving Agent System. arXiv:2511.10395 [cs.LG] <https://arxiv.org/abs/2511.10395>
- [41] Guibin Zhang, Muxin Fu, Guancheng Wan, Miao Yu, Kun Wang, and Shuicheng Yan. 2025. G-Memory: Tracing Hierarchical Memory for Multi-Agent Systems. arXiv:2506.07398 [cs.MA] <https://arxiv.org/abs/2506.07398>
- [42] Guibin Zhang, Haotian Ren, Chong Zhan, Zhenhong Zhou, Junhao Wang, He Zhu, Wangchunshu Zhou, and Shuicheng Yan. 2025. MemEvolve: Meta-Evolution of Agent Memory Systems. arXiv:2512.18746 [cs.CL] <https://arxiv.org/abs/2512.18746>
- [43] Kai Zhang, Xiangchao Chen, Bo Liu, Tianci Xue, Zeyi Liao, Zhihan Liu, Xiyao Wang, Yuting Ning, Zhaorun Chen, Xiaohan Fu, Jian Xie, Yuxuan Sun, Boyao Gou, Qi Qi, Zihang Meng, Jianwei Yang, Ning Zhang, Xian Li, Ashish Shah, Dat Huynh, Hengduo Li, Zi Yang, Sara Cao, Lawrence Jang, Shuyan Zhou, Jiacheng Zhu, Huan Sun, Jason Weston, Yu Su, and Yifan Wu. 2025. Agent Learning via Early Experience. arXiv:2510.08558 [cs.AI] <https://arxiv.org/abs/2510.08558>
- [44] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. 2025. Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models. arXiv:2510.04618 [cs.LG] <https://arxiv.org/abs/2510.04618>
- [45] Shuyu Zhang, Yujie Liu, Xinru Wang, Cheng Zhang, Yanmin Zhu, and Bin Li. 2026. DarwinTOD: LLM Driven Lifelong Self Evolution for Task Oriented Dialog Systems. arXiv:2601.07248 [cs.MA] <https://arxiv.org/abs/2601.07248>
- [46] Shengtao Zhang, Jiaqian Wang, Ruiwen Zhou, Junwei Liao, Yuchen Feng, Weinan Zhang, Ying Wen, Zhiyu Li, Feiyu Xiong, Yutao Qi, Bo Tang, and Muning Wen. 2026. MemRL: Self-Evolving Agents via Runtime Reinforcement Learning on Episodic Memory. arXiv:2601.03192 [cs.CL] <https://arxiv.org/abs/2601.03192>
- [47] Zeyu Zhang, Quanyu Dai, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. 2025. A Survey on the Memory Mechanism of Large Language Model-based Agents. *ACM Trans. Inf. Syst.* 43, 6, Article 155 (Sept. 2025), 47 pages. doi:10.1145/3748302
- [48] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL: LLM agents are experiential learners. In *Proceedings of the Thirty-Eighth AAAI Conference on Artificial Intelligence and Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence and Fourteenth Symposium on Educational Advances in Artificial Intelligence (AAAI'24/IAAI'24/EAAI'24)*. AAAI Press, Article 2188, 11 pages. doi:10.1609/aaai.v38i17.29936
- [49] Longtao Zheng, Rundong Wang, Xinrun Wang, and Bo An. 2024. Synapse: Trajectory-as-Exemplar Prompting with Memory for Computer Control. arXiv:2306.07863 [cs.AI] <https://arxiv.org/abs/2306.07863>

A Algorithmic Details

Algorithm 2 details the procedure for constructing structured experience hints from a ranked set of retrieved cases under a fixed budget. This design ensures that limited prompt capacity is allocated preferentially to experience instances that convey concrete corrective knowledge, while retaining flexibility to incorporate additional relevant cases when budget allows.

B Datasets and Models

B.1 Datasets

HumanEval is a standard benchmark for evaluating functional program synthesis by large language models. It consists of 164 hand-written Python programming tasks, each defined by a natural language description, a function signature, and a set of hidden unit tests used for automatic evaluation. The benchmark focuses on

Algorithm 2 Hint Construction from Ranked Cases.

Require: Ranked cases C_{rank} , correction relation $\mathcal{E}_{\text{fix}}$ (fixed_by), hint budget M

Ensure: Structured hint set $\mathcal{H}$

```

1:  $\mathcal{S} \leftarrow []$  ▷ selected cases for hint construction
2:  $\mathcal{W}_{\text{fix}} \leftarrow []$  ▷ warning cases with a fixed_by relation
3: for all  $c \in C_{\text{rank}}$  do
4:   if  $c$  is a warning case and  $\exists(c \rightarrow c^+) \in \mathcal{E}_{\text{fix}}$  then
5:     append  $c$  to  $\mathcal{W}_{\text{fix}}$ 
6:     if  $|\mathcal{W}_{\text{fix}}|$  reaches the configured limit then
7:       break
8:   end if
9: end if
10: end for
11: for all  $c \in \mathcal{W}_{\text{fix}}$  do
12:   add  $c$  to  $\mathcal{S}$ 
13:   if  $|\mathcal{S}| \geq M$  then
14:     return FormatHint( $\mathcal{S}, \mathcal{E}_{\text{fix}}$ )
15:   end if
16: end for
17: if including corrected counterparts then
18:   for all  $c \in \mathcal{W}_{\text{fix}}$  do
19:      $c^+ \leftarrow$  the unique case such that  $(c \rightarrow c^+) \in \mathcal{E}_{\text{fix}}$ 
20:     if  $c^+$  exists then
21:       add  $c^+$  to  $\mathcal{S}$ 
22:     end if
23:     if  $|\mathcal{S}| \geq M$  then
24:       return FormatHint( $\mathcal{S}, \mathcal{E}_{\text{fix}}$ )
25:     end if
26:   end for
27: end if
28: for all  $c \in C_{\text{rank}}$  do
29:   add  $c$  to  $\mathcal{S}$  ▷ deduplicate while adding
30:   if  $|\mathcal{S}| \geq M$  then
31:     break
32:   end if
33: end for
34:  $\mathcal{H} \leftarrow$  FormatHint( $\mathcal{S}, \mathcal{E}_{\text{fix}}$ )
35: return  $\mathcal{H}$ 

```

semantic correctness rather than surface-level similarity, as generated programs must pass all test cases to be considered correct. HumanEval is particularly suitable for studying experience reuse, since many tasks share recurring programming patterns and failure modes, allowing prior successes and corrections to inform subsequent code generation attempts.

EvalPlus is an enhanced evaluation suite built upon HumanEval, designed to provide stricter and more comprehensive correctness verification. It augments each original problem with additional test cases, including adversarial and edge-case inputs, to reduce false positives caused by insufficient test coverage. In this work, EvalPlus is used to assess whether the benefits of experience reuse persist under more rigorous functional validation, ensuring that observed improvements reflect genuine semantic correctness rather than overfitting to limited tests.

MuSiQue (Multi-hop Sequential Question Answering) is a benchmark designed to evaluate compositional multi-hop reasoning over text. Each question is constructed to require a specific reasoning chain involving multiple supporting facts, often combining entity linking, relational reasoning, and temporal inference. By explicitly controlling reasoning depth and discouraging shortcut solutions, MuSiQue provides a challenging testbed for analyzing whether accumulated experience can be abstracted into reusable reasoning patterns across structurally similar multi-hop questions.

HotpotQA is a large-scale multi-hop question answering dataset built from Wikipedia articles, containing over 100k question-answer pairs. A key characteristic of HotpotQA is the annotation of sentence-level supporting facts, enabling supervision and evaluation of explainable reasoning. The dataset includes both bridge-type questions, which require identifying intermediate entities, and comparison-type questions that integrate information across documents. In our experiments, HotpotQA serves to evaluate whether experience graphs can capture and reuse structural reasoning signals, such as missing hops or incorrect entity associations, in complex multi-document inference tasks.

B.2 Models

All models used in this work are accessed exclusively through APIs and model selection is guided by dataset difficulty, task specialization, and a balanced consideration of open-weight versus proprietary models.

Qwen3-1.7B is a small-scale open-weight model in the Qwen3 family, designed for efficient inference under strict computational constraints. We include Qwen3-1.7B to examine whether structured experience reuse remains effective at very limited model capacity, providing a lower-bound assessment of EXG’s robustness.

Qwen3-8B is a compact open-weight model in the Qwen3 family, with approximately 8B parameters, designed to provide strong general reasoning capabilities under constrained computational budgets. In our experiments, Qwen3-8B is primarily used to assess the robustness of experience reuse mechanisms under limited model capacity.

Qwen3-14B is a mid-to-large scale open-weight model in the Qwen3 family, positioned between compact models such as Qwen3-8B and larger-capacity models like Qwen-Max. It offers stronger reasoning and representation capacity than smaller Qwen3 variants

while remaining more computationally efficient than frontier-scale models. In our experiments, Qwen3-14B is used to evaluate the effectiveness of structured experience reuse at an intermediate model scale, serving as a bridge between lightweight and high-capacity backbones.

Qwen3-Coder-Flash is a lightweight, code-oriented model optimized for fast inference and program synthesis tasks. It has an approximate parameter scale of $\sim 30\text{B}$ and is designed to efficiently handle function-level code generation with low latency. In our experiments, Qwen3-Coder-Flash is mainly used on HumanEval and EvalPlus, where rapid iteration and clear execution feedback are essential.

Qwen-Plus is a mid-sized general-purpose language model that offers a balance between reasoning capability and computational efficiency, making it suitable for reasoning-intensive tasks. In our experiments, Qwen-Plus is used primarily for question answering benchmarks, including MuSiQue and HotpotQA.

Qwen-Max is a large-capacity model in the Qwen3 family and it provides stronger reasoning and comprehension abilities, particularly for complex language understanding and multi-hop inference. In our experiments, Qwen-Max is used for multi-hop question answering tasks, including MuSiQue and HotpotQA.

C Additional Experimental Analysis

C.1 Online Token Usage

To better understand how structured experience reuse affects computational cost, we analyze token consumption across different methods and datasets, focusing on how input and output tokens are redistributed under a fixed interaction budget.

On HumanEval, baseline methods without structured experience reuse exhibit substantial token overhead due to repeated full-code generation. Reflexion consumes 117,378 total tokens across all tasks, whereas EXG consumes 125,304 tokens, corresponding to a modest increase of 6.8% in total token usage. This increase is driven by

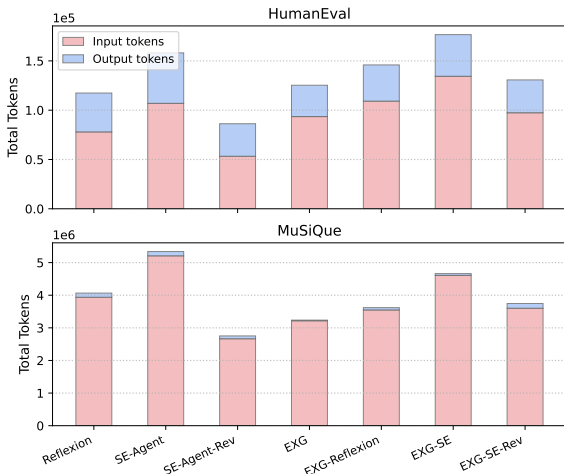


Figure 7: Token usage breakdown on HumanEval and MuSiQue. Each bar shows the total number of tokens consumed, with input tokens stacked below output tokens.

a deliberate rise in input tokens (+20.0%), reflecting the injection of experience hints, while output tokens are reduced by 19.3%. Compared to SE-Agent-Lite, which incurs 158,125 total tokens, EXG reduces total token consumption by 20.8%, with a 37.8% reduction in output tokens. These results indicate that, on HumanEval, EXG substantially reduces redundant generations, and that its slightly higher prompt cost is more than offset when compared to retry- and revision-based baselines.

The impact of experience reuse is significantly amplified on MuSiQue, a long-context multi-hop reasoning benchmark. Reflexion consumes 4.07M total tokens, while EXG reduces this to 3.24M tokens, yielding a 20.4% reduction in total token usage. This improvement is almost entirely attributable to a dramatic reduction in output tokens (84.0%), despite an 18.3% increase in input tokens. Relative to SE-Agent-Lite, which exceeds 5.34M total tokens, EXG achieves an even larger reduction of 39.4%. These figures highlight that, in long-horizon reasoning tasks, avoiding repeated reasoning chains yields substantial absolute token savings.

Across both datasets, EXG consistently shifts computation from output-heavy repeated generation to input-side structured guidance. While this shift leads to moderate increases in prompt length, it results in large and consistent reductions in output tokens, ranging from 19.3% on HumanEval to over 80% on MuSiQue. The magnitude of output-token reduction scales with task complexity, demonstrating that the efficiency gains of EXG arise from reduced redundant exploration rather than dataset-specific effects.

Taken together, these results show that EXG improves token efficiency not by compressing individual prompts or enforcing shorter outputs, but by altering the interaction dynamics of self-evolving agents. By leveraging accumulated experience to guide generation earlier, EXG reduces repeated attempts and enables experience to compound over time, yielding superior accuracy–efficiency trade-offs, especially in long-context reasoning settings.

C.2 Online Learning Curve

On HumanEval, all methods start from a similar performance level in the early stage of deployment. After the first 20 tasks, cumulative Pass@1 for Reflexion, SE-Agent, and EXG-based methods all fall within a narrow range of approximately 50–55%. As more tasks are observed, baseline methods exhibit only marginal improvement: after 60 tasks, Reflexion remains around 56–58% and shows little further increase. In contrast, EXG-based methods demonstrate a sustained upward trend. By 60 tasks, EXG-based methods reach approximately 68–70%, corresponding to an absolute improvement of about 14–15 percentage points over its early-stage performance. This gap continues to widen as deployment proceeds. By the end of the task sequence, EXG-based methods attain a cumulative Pass@1 of approximately 80%, while baseline methods plateau near 55–58%. Overall, EXG improves Pass@1 by roughly 25 percentage points from early to late stages, whereas baselines improve by less than 5 points.

A similar but more pronounced pattern is observed for Pass@2. At the early stage (20 tasks), all methods achieve comparable Pass@2 values of approximately 72–76%. Baseline methods quickly saturate: Reflexion stabilizes near 75–78% after 40–60 tasks, with negligible improvement thereafter. In contrast, EXG-based methods continue

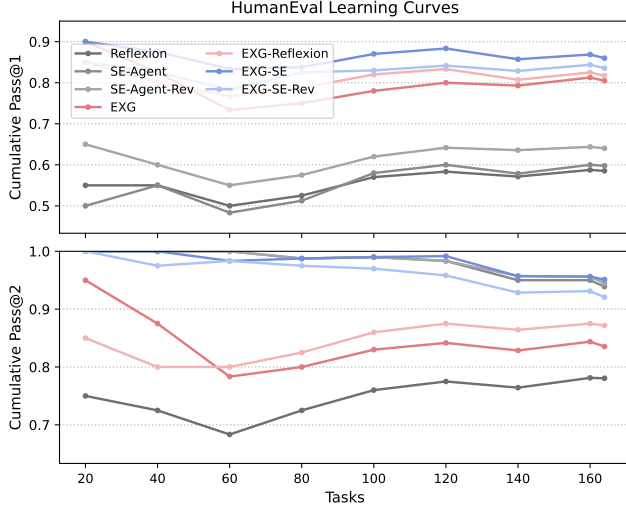


Figure 8: Learning curves on HumanEval.

to improve as more tasks are seen. By around 60 tasks, EXG-based methods reach approximately 85%, already exceeding baseline performance by about 7–10 percentage points. By the end of the task sequence, EXG-based methods achieve a cumulative Pass@2 close to 90%, compared to 75–78% for baseline methods. This corresponds to an absolute late-stage improvement of roughly 12–15 percentage points over Reflexion, indicating that experience reuse compounds the effectiveness of limited retries over time.

MuSiQue presents a more challenging setting in which early performance remains low for all methods, reflecting the difficulty of long-context, multi-hop reasoning. During the initial phase, cumulative Pass@1 for both baseline methods and EXG-based methods remains clustered in a narrow band of roughly 30–35%, suggesting that little reusable structure is available at the beginning. As task

exposure increases, baseline methods exhibit limited progression: even after around 60 tasks, their Pass@1 rises only marginally to approximately 36–38%, after which further gains largely diminish. In contrast, EXG-based methods collectively display a qualitatively different trajectory. The core EXG-based method shows a consistent upward trend as experience accumulates, reaching approximately 45–47% Pass@1 by 60 tasks—an absolute gain of about 10–12 percentage points relative to its early-stage performance. Variants that combine EXG with additional self-evolution mechanisms (e.g., reflection or revision) follow a similar but slightly more variable pattern, indicating that the primary driver of improvement is the experience graph itself rather than auxiliary operators. By the end of the task sequence, EXG-based methods approach 50% Pass@1, while baseline methods remain below 40%, yielding a late-stage advantage of approximately 12–15 percentage points.

Allowing a second attempt further accentuates the separation between baseline and EXG-based methods. In the early stage, Pass@2 values for all methods lie in a comparable range of approximately 45–50%, indicating that limited retries alone do not overcome the inherent difficulty of the task. Baseline methods quickly saturate: Reflexion stabilizes around 50–52% after 40–60 tasks, with little subsequent improvement. By contrast, EXG-based methods continue to benefit from accumulated experience throughout deployment. The core EXG method reaches approximately 60–62% Pass@2 by 60 tasks, exceeding baseline performance by 8–10 percentage points at the same stage. EXG-based variants follow closely, with minor fluctuations attributable to their additional control logic. By the end of deployment, EXG-based methods achieve cumulative Pass@2 values near 65%, compared to 50–52% for baseline methods, corresponding to a late-stage improvement of approximately 13–15 percentage points. These results indicate that structured experience reuse not only improves first-attempt reasoning, but also increasingly enhances the effectiveness of limited retries in long-horizon multi-hop settings.

C.3 Graph Statistics

Table 4 provides quantitative evidence on how the experience graph evolves across datasets. On both HumanEval and MuSiQue, the number of case nodes closely tracks the number of processed tasks, with an average of around 95–100 cases on HumanEval and around 400–500 cases on MuSiQue across EXG-based methods. This confirms that graph growth is approximately linear in the number of tasks rather than in the number of attempts, despite cases being defined at the attempt level. As a result, the experience graph remains compact and task-aligned throughout deployment, avoiding uncontrolled expansion due to repeated retries.

Beyond graph size, the two datasets exhibit a strikingly similar local connectivity pattern. After collapsing symmetric similarity relations into undirected edges, each case is connected to around 18–20 similar cases on average, even though the absolute number of cases differs by nearly a factor of five. This consistency indicates that EXG induces a stable similarity backbone that is largely independent of dataset scale or domain. Such dense local neighborhoods allow experience to propagate across related cases, providing a structural explanation for the steadily improving learning curves observed in both code generation and multi-hop reasoning tasks.

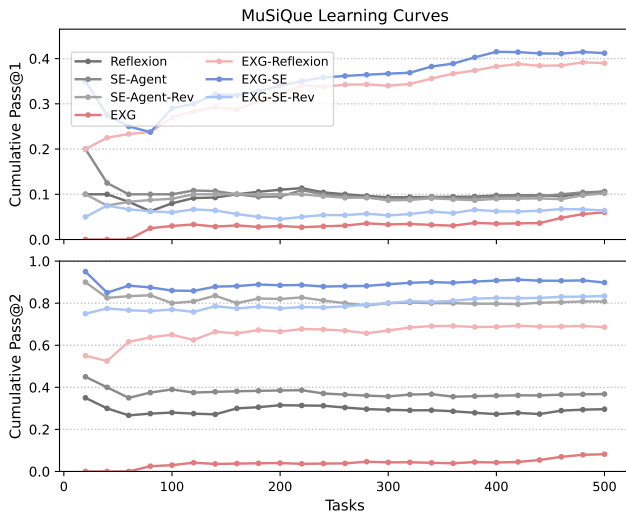


Figure 9: Learning curves on MuSiQue.

Table 4: Experience graph statistics averaged over task-level graph snapshots.

Method	HumanEval			MuSiQue		
	Cases	<i>similar_to</i>	<i>fixed_by</i>	Cases	<i>similar_to</i>	<i>fixed_by</i>
EXG	99.9	905	3.6	491.8	5,021	2.9
EXG-Reflexion	98.1	880	3.4	412.4	3,937	81.3
EXG-Revision	96.5	876	10.4	486.0	4,692	187.1
EXG-SE	93.9	843	9.5	406.4	3,926	129.7

In contrast, correction relations exhibit substantial variation across datasets and EXG-based methods. On HumanEval, the number of *fixed_by* edges remains consistently low, ranging from around 3 to 10, corresponding to roughly 3–11% of cases participating in explicit correction relations. On MuSiQue, however, the number of *fixed_by* edges varies much more widely, from around 3 for the core EXG method to over 180 when combined with reflection- or revision-based strategies. This increase is particularly pronounced for reflection-based architectures, as explicit reflection after a failed attempt makes it easier to align a *warning* case with its corrected

golden counterpart, thereby facilitating the construction of *fixed_by* relations. As a result, while similarity relations form a stable backbone for experience propagation, correction relations are more sensitive to auxiliary self-evolution mechanisms. Crucially, even when reflection substantially increases the number of *fixed_by* edges, these relations remain sparse relative to overall graph size, indicating that EXG-based methods rely on amplifying a limited number of well-aligned, high-impact fixes rather than on frequent trial-and-error corrections.